PROJECT REPORT

Name : Harikarthikeyan v

Python Fundamentals – Module End Assignment 3

Survey Feedback Analyzer

1. Project Overview

The Survey Feedback Analyzer is a Python-based project designed to collect, clean, and analyze textual feedback provided by users. The project uses fundamental Python programming concepts to process survey responses and extract meaningful insights.

The program stores survey information in a dictionary of lists containing serial numbers, names, written feedback, and ratings. It allows users to add new feedback entries, cleans the text, counts feedback entries containing specified words, calculates the average rating, identifies the longest comment, and generates a list of unique words.

2. Project Objectives

- To understand and implement Python dictionaries and lists.
- To collect additional survey feedback using user input.
- To apply loops and conditional statements for data processing.
- To clean textual feedback by removing punctuation and unnecessary spaces.
- To convert feedback text into lowercase for consistent analysis.
- To create user-defined functions for word-count analysis.
- To calculate the average survey rating.
- To identify the longest feedback comment based on word count.
- To generate a list of unique words from all feedback entries.
- To sort feedback entries by rating using Python's built-in functions.

3. Problem Statement

Analyzing textual feedback is important for understanding user experiences, identifying service-related issues, and improving customer satisfaction. However, raw feedback often contains inconsistent capitalization, unnecessary spaces, and punctuation that can affect text processing.

This project aims to develop a simple Python program that organizes survey feedback, performs basic text cleaning, and extracts useful information using loops, conditional statements, string operations, and user-defined functions.

4. Tools and Technologies Used

- **Programming Language:** Python
- **Development Environment:** Jupyter Notebook
- **Data Structures:** Dictionaries, lists, and sets
- **Python Concepts:** Loops, conditional statements, functions, string manipulation, and exception handling
- **Built-in Functions:** print(), input(), len(), sum(), sorted(), zip(), and range()

5. Dataset Description

The project uses a predefined dataset containing 10 survey feedback entries. Each entry includes a serial number, respondent name, written feedback, and rating.

The dataset is stored in a dictionary named feedback_data.

Column	Description
S_No	Unique serial number for each feedback entry
Name	Name of the respondent
Feedback	Written comment provided by the respondent
Rating	Survey rating on a scale of 1 to 5

6. Project Implementation

6.1 Step 1: Preloaded Feedback Data

The project begins by creating a dictionary of lists containing the 10 predefined feedback entries.

The dictionary organizes all survey information into four corresponding lists: S_No, Name, Feedback, and Rating.

Outcome: The initial survey data is successfully stored and displayed.

6.2 Step 2: Adding More Feedback Entries

The program asks the user how many additional feedback entries they want to add. For every new entry, it collects the respondent's name, written feedback, and rating.

A for loop processes the requested number of entries. The append() method adds the new information to the corresponding lists, while len() is used to generate the next serial number. A while loop and conditional statements validate that ratings are between 1 and 5.

Outcome: The dataset can be expanded dynamically through user input.

6.3 Step 3: Text Cleaning

The feedback entries are cleaned using Python string operations.

The following operations are performed:

- Remove periods, commas, exclamation marks, and question marks.
- Replace multiple spaces with a single space.
- Remove leading and trailing spaces.
- Convert all feedback text to lowercase.

The .replace() method removes the specified punctuation, while .split() and ' '.join() help normalize spacing. The .lower() method converts the text into lowercase.

Outcome: Feedback entries are standardized for consistent word analysis.

6.4 Step 4: Word Count Insights

A user-defined function named count_word_in_feedbacks(word) is created to count the number of feedback entries containing a specified word.

The function checks each cleaned comment, splits it into individual words, and compares those words with the requested word. The count increases whenever a matching entry is found.

The function is used to count feedback entries containing:

- good
- poor
- excellent

Outcome: The program identifies how many feedback entries contain each specified word.

6.5 Step 5: Final Summary and Insights

The final analysis consists of four main operations.

A. Displaying the cleaned dataset

The program displays the final dictionary, including serial numbers, names, cleaned feedback, and ratings.

B. Calculating the average rating

The average rating is calculated by dividing the sum of all ratings by the total number of ratings.

Formula:

Average Rating = Total of Ratings / Number of Ratings

C. Identifying the longest feedback

The program calculates the word count of every comment and identifies the comment with the highest word count. It also displays the respondent's name and the corresponding feedback.

D. Generating unique words

A set is used to collect the words appearing across all feedback entries. Since sets automatically eliminate duplicates, the resulting collection contains only unique words. The sorted() function displays these words in alphabetical order.

Outcome: The program produces a summary of the cleaned data and extracts useful information from the survey responses.

6.6 Optional Step: Sorting Feedback by Rating

The project can optionally sort feedback entries from the highest rating to the lowest rating.

The zip() function combines the related values from the four lists, and sorted() arranges the combined records using the rating as the sorting key.

Outcome: Feedback entries can be viewed in descending order of rating.

7. Python Concepts Applied

Concept	Application
Dictionary	Stores survey data using named keys
Lists	Store multiple values for each field
input()	Collects additional feedback from users
for loop	Iterates through feedback entries
while loop	Validates user ratings
Conditional statements	Check ratings and identify matching words
Functions	Perform reusable word-count analysis
String methods	Clean and standardize feedback
Set	Stores unique words
sum() and len()	Calculate the average rating
zip() and sorted()	Sort feedback entries by rating
Exception handling	Handles invalid numeric rating input

8. Expected Results

After successful execution, the program should produce the following outputs:

- Display the initial 10 feedback entries.
- Accept and append additional feedback entries.
- Generate sequential serial numbers for new entries.
- Display cleaned feedback with standardized spacing and lowercase text.
- Count feedback entries containing good, poor, and excellent.
- Display the final cleaned dictionary of lists.
- Calculate and display the average rating.
- Identify the longest comment by word count.
- Display the unique words used across all feedback entries.

- Optionally display feedback entries sorted by rating.

For the original 10 feedback entries, the expected average rating is **3.40 out of 5**. The word-count results for the original dataset are 3 entries containing good, 2 containing poor, and 1 containing excellent.

These numerical results apply to the original dataset. Adding more feedback entries may change the results.

9. Testing and Validation

The following checks can be performed to verify the program:

1. Confirm that the initial dictionary contains 10 records.
2. Add new feedback entries and verify that all four lists have the same number of elements.
3. Enter invalid ratings and verify that the program requests a valid rating between 1 and 5.
4. Check that the text-cleaning operations remove the specified punctuation and extra spaces.
5. Verify that the word-count function returns the correct number of matching feedback entries.
6. Compare the calculated average rating with the sum of ratings divided by their count.
7. Verify that the longest feedback is selected according to its number of words.
8. Confirm that the unique-word collection contains no duplicate entries.
9. Check that optional sorting displays ratings from highest to lowest.

10. Learning Outcomes

Through this project, the following skills are developed:

- Understanding Python data structures and their practical applications.
- Collecting and managing user input.
- Applying loops and conditional logic to process data.
- Performing basic text cleaning and normalization.

- Developing and calling user-defined functions.
- Calculating summary statistics using built-in functions.
- Identifying unique values using sets.
- Organizing related data and sorting records.
- Testing program logic and validating output.

11. Conclusion

The Survey Feedback Analyzer demonstrates how fundamental Python programming concepts can be applied to a practical data-processing problem. The project organizes survey responses, accepts additional user input, standardizes textual feedback, and extracts information through word counting and summary calculations.

By completing this assignment, the core concepts of dictionaries, lists, loops, conditional statements, functions, string manipulation, and sets are applied in a single project. The resulting program provides a simple foundation for more advanced feedback analysis and data analytics applications.

Project Status: Completed after successful execution and verification of all required steps.